

Azure SQL Database

Relational fundamentals, architecture, indexing, scaling, security, recovery, integration, and production

Curated study notes for the RAG learning project. Synthesized from official documentation; verify current product behavior in official docs before production use.

1. Azure SQL Database Overview

Azure SQL Database is a fully managed relational database service based on the SQL Server database engine. It is a PaaS service designed for modern cloud applications and managed relational workloads.

Key idea: Azure SQL Database is a managed database service; Microsoft handles much of the underlying infrastructure, patching, backups, and high-availability operations.

What it is

Azure SQL Database provides a relational SQL engine in Azure with familiar T-SQL, security features, indexing, transactions, and management capabilities.

- Good fit for cloud applications.
- Supports single databases and elastic pools among other deployment choices.
- Hyperscale provides a different architecture for large-scale workloads.

Why managed SQL

Managed services reduce infrastructure administration so teams can focus on schema, queries, applications, and data.

- Automatic maintenance reduces operational burden.
- Scaling options support changing workloads.
- Built-in security and monitoring features support enterprise use.

2. Architecture, Logical Server, and Compute

Azure SQL Database separates the logical database service from the underlying infrastructure. A logical server can host databases and provide a management boundary, while databases consume compute/storage resources according to their service tier.

Key idea: Do not confuse a logical server with a traditional SQL Server VM. Azure SQL Database abstracts the operating system and database host management.

Logical server

A logical server provides a management and connectivity boundary for databases. It can host multiple databases and apply server-level settings and firewall/network controls.

- Use naming and environment boundaries deliberately.
- Manage administrator and identity access securely.

Compute and service tiers

Azure SQL Database offers different purchasing/service options and compute models. Exact tiers and features evolve, so use current Azure documentation for production sizing.

- Choose based on CPU, memory, I/O, concurrency, and availability requirements.
- Serverless can fit intermittent workloads.
- Elastic pools can share resources across multiple databases.

Storage

Azure manages storage, availability, and many infrastructure operations. Hyperscale uses a highly scalable storage architecture with separate compute/storage concepts.

- Application design still determines logical data size and indexing needs.

3. T-SQL, Transactions, and Relational Design

Azure SQL Database uses the SQL Server engine and supports standard relational concepts such as tables, constraints, transactions, views, stored procedures, and indexes.

Key idea: Cloud hosting does not remove relational fundamentals. Good keys, constraints, transactions, and query design remain essential.

Transactions

Transactions group operations into a logical unit. ACID properties help maintain correctness in concurrent systems.

- Keep transactions appropriately scoped.
- Avoid holding locks longer than necessary.
- Understand isolation levels and concurrency behavior.

Constraints

Primary keys, foreign keys, unique constraints, check constraints, and defaults help enforce data quality at the database layer.

- Use constraints when the database is the right enforcement boundary.
- Do not rely only on application validation for critical invariants.

Stored procedures and views

Views provide reusable query abstractions; stored procedures encapsulate procedural database logic and can centralize operations.

- Keep interfaces documented.
- Monitor query behavior of frequently called procedures.

4. Indexing and Query Performance

Indexes improve data access for appropriate query patterns but add storage and write overhead. Query tuning should start with actual execution behavior.

Key idea: An index is useful when it supports a common access pattern; more indexes are not automatically better.

Clustered and nonclustered indexes

A clustered index determines the table's primary physical/logical ordering strategy; nonclustered indexes provide additional access paths.

- Choose keys based on workload.
- Avoid very wide indexes without a reason.
- Covering indexes can reduce lookups for selected queries.

Statistics

Statistics describe data distribution and help the optimizer estimate cardinalities.

- Stale or misleading statistics can produce poor plans.
- Parameter-sensitive workloads may need special investigation.

Execution plans

Plans show scans, seeks, joins, sorts, lookups, and estimated/actual row counts.

- Look for large mismatches between estimated and actual rows.
- Tune the root cause rather than adding indexes blindly.

5. Scaling, Serverless, Elastic Pools, and Hyperscale

Azure SQL Database offers multiple ways to scale or share resources. The right option depends on workload shape, database count, size, and predictability.

Key idea: Scaling is a workload decision: scale up for more resources per database, scale across databases with pools, or choose architectures designed for very large databases.

Serverless

Serverless compute can automatically scale compute for intermittent workloads and can pause after inactivity depending on configuration.

- Useful for variable usage.
- Understand resume latency and billing behavior.

Elastic pools

Elastic pools provide shared resources for multiple databases whose individual peaks occur at different times.

- Useful for SaaS patterns with many databases.
- Right-size based on aggregate workload.

Hyperscale

Hyperscale is designed for large databases and separates compute and storage in its architecture, enabling rapid scaling characteristics.

- Use when database size and throughput requirements justify it.
- Read scale and geo-replication capabilities depend on the architecture and configuration.

6. Security and Identity

Azure SQL Database provides layered security using authentication, authorization, networking, encryption, auditing, threat detection, and data protection features.

Key idea: Security design should combine identity, network boundaries, least privilege, encryption, and auditing.

Authentication

Azure SQL supports Microsoft Entra authentication alongside SQL authentication options. Identity-based access is often preferred for cloud-native applications.

- Use managed identities where supported.
- Avoid long-lived passwords in application configuration.

Authorization

Database roles and permissions control access to objects. Fine-grained controls can support separation of duties.

- Grant least privilege.
- Use groups/roles for maintainability.
- Separate application runtime rights from administrator rights.

Encryption and protection

Encryption at rest, TLS in transit, transparent data encryption, dynamic data masking, row-level security, and threat-protection capabilities can address different risks.

- Choose controls based on threat model and compliance requirements.
- Customer-managed keys introduce key-management responsibilities.

Networking

Firewall rules, private endpoints, and network controls can restrict access paths.

- Prefer private access for sensitive workloads where required.
- Document allowed application network paths.

7. Backup, High Availability, and Disaster Recovery

Azure SQL Database is a managed service with built-in high availability and automated backup capabilities. Disaster recovery design still requires explicit recovery objectives and testing.

Key idea: Backup availability is not the same as a complete disaster-recovery plan. Define RPO, RTO, and regional-failure behavior.

Backups

Azure SQL Database performs automated backups and supports point-in-time restore capabilities subject to service configuration and retention policies.

- Know the retention window for the chosen service.
- Test restores rather than assuming backups work.

High availability

The service manages infrastructure redundancy to support availability. Application design should still handle transient failures and reconnect appropriately.

- Use retry policies carefully.
- Do not retry non-transient errors indefinitely.

Geo-replication and failover

Business continuity options can replicate databases to other regions and support failover scenarios.

- Define which region is secondary.
- Understand DNS/application failover behavior.
- Test failover procedures.

Recovery objectives

RPO describes acceptable data loss; RTO describes acceptable recovery time.

- Write objectives before choosing architecture.
- Measure recovery procedures against them.

8. Data Integration and Analytics Patterns

Azure SQL Database often participates in broader data platforms, serving operational applications while data is copied or transformed into analytical stores.

Key idea: Do not overload an OLTP database with large analytical scans when a separate analytical workload is more appropriate.

Operational role

Azure SQL Database is well suited to transactional application data and structured business systems.

- Use transactions and constraints.
- Tune for application concurrency.

Analytical integration

ADF, Databricks, Synapse, and other services can extract or transform SQL data for analytics.

- Prefer incremental extraction where possible.
- Track source changes and watermarks.
- Avoid impacting production databases with uncontrolled extraction.

Serving

Curated analytical data can be served through a warehouse, lakehouse, or semantic layer depending on requirements.

- Keep source-of-truth ownership clear.
- Document freshness expectations.

9. Common SQL Database Design Problems

Many database incidents arise from predictable design issues: poor indexing, unbounded queries, blocking, missing constraints, bad data types, and uncontrolled growth.

Key idea: A production database is both a correctness system and a performance system.

Blocking and contention

Concurrent transactions can block each other. Long-running transactions increase lock duration and can create cascading waits.

- Keep transactions short.
- Investigate blocking chains.
- Choose isolation behavior deliberately.

Bad data types

Oversized strings, incorrect numeric precision, or storing structured data as untyped text can increase storage and complicate queries.

- Use appropriate numeric precision.
- Use date/time types for temporal data.
- Avoid storing everything as varchar.

Missing constraints

Without keys and constraints, duplicate or orphaned data can spread through the system.

- Define primary keys.
- Use uniqueness constraints where business rules require them.
- Use foreign keys when referential integrity belongs in the database.

Growth

Tables, indexes, logs, and backups grow over time.

- Monitor storage and performance.
- Archive or purge according to documented retention policies.

10. Azure SQL Interview and Production Checklist

Use this page as a revision checklist for Azure SQL Database and as a bridge between relational database knowledge and Azure data engineering.

Key idea: Know both the database fundamentals and the Azure-managed-service differences.

Core questions

Be able to explain logical servers, databases, service tiers, indexes, transactions, execution plans, backups, and authentication.

- Why use Azure SQL Database instead of SQL Server on a VM?
- When would you use Hyperscale?
- What problem does an elastic pool solve?
- How do indexes affect reads and writes?

Production checklist

Define workload, size, concurrency, RPO/RTO, security, networking, monitoring, and cost before selecting a tier.

- Use identity-based access.
- Monitor query performance.
- Test restores and failover.
- Document ownership and escalation paths.

Data engineering connection

Azure SQL often acts as a source or serving store inside a larger pipeline.

- ADF can orchestrate ingestion.
- Databricks/Spark can transform at scale.
- Synapse can provide analytical capabilities.
- A lake/lakehouse can retain large historical datasets.

Selected official sources

- <https://learn.microsoft.com/en-us/azure/azure-sql/?view=azuresql>
- <https://learn.microsoft.com/en-us/azure/azure-sql/azure-sql-iaas-vs-paas-what-is-overview?view=azuresql>
- <https://learn.microsoft.com/en-us/azure/azure-sql/database/features-comparison?view=azuresql>
- <https://learn.microsoft.com/en-us/azure/azure-sql/database/service-tier-hyperscale-frequently-asked-questions-faq?view=azuresql>